



Introduction to HashMap & HashSet - Iterate
over map

Given queries, find frequency of an element

No. of Distinct Elements

Pair sum

Count of Pair Sum

Good eve

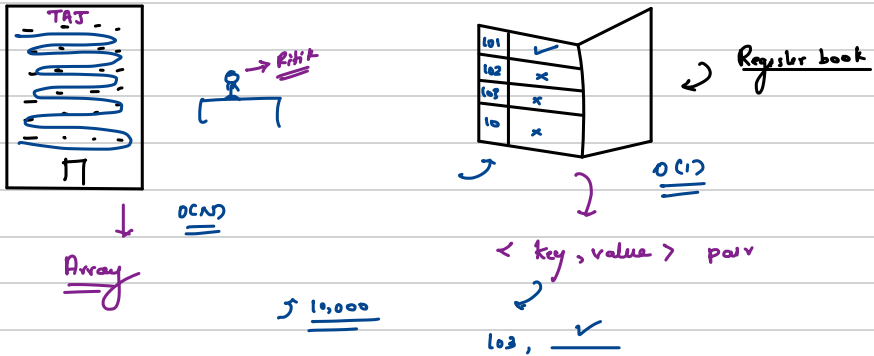
9:06pm ← lets.

start

<https://discord.com/invite/mH2Pj4Vx3X>



Join discord group.



Clavorn → Attendance

HashMap < key, value >

key → rollNo

value → present, absent

HashMap will you use to store the population of every country

country → string

population → long

HashMap will you use to store the no of states of every country?

country → "India"

states → 29

HashMap < string, int > statesOfCountry = new HashMap < >();

HashMap will you use to store the name of all states of every country?

country $\rightarrow$ "India" $\rightarrow$ string
state $\rightarrow$ ["____", "____", "____", ...]
 $\downarrow$
list of strings.

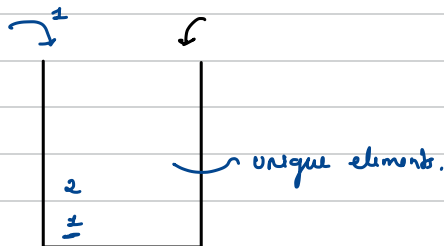
HashMap < string, List < string > > name of state of country.

$\rightarrow$ HashMap will you use to store the population of each state in every country?

country $\rightarrow$ "India"
state population $\rightarrow$ "Goa" $\rightarrow$ 2
"Maharashtra" $\rightarrow$ _
"Delhi" $\rightarrow$ _

HashMap < string, HashMap < string, long > >

HashSet



HashSet < key > .

HashMap

INSERT(Key, Value): new key-value pair is inserted. If the key already exists, it does no change.

SIZE: returns the number of keys.

DELETE(Key): delete the key-value pair for given key.

UPDATE(Key, Value): previous value associated with the key is overridden by the new value.

SEARCH(Key): searches for the specified key.

HashSet

INSERT(Key): inserts a new key. If key already exists, it does no change.

SIZE: returns number of keys.

DELETE(Key): deletes the given key.

SEARCH(Key): searches for the specified key.

python

dict $\rightarrow$ key value

set

JS

Map

set

C#

dict

hashset.

T.C: $O(1)$



Time complexity for all above operation.

Given N elements and Q queries, find the frequency of the elements provided in a query.

 $N=10$

A = { 2, 6, 3, 8, 2, 8, 2, 3, 8, 10, 6 }

$$q =$$

element	freq
2	3
8	3
3	2
5	0

Brute force . for each Query 'Q' traverse the entire array and count the freq.

$T_C: O(Q \times N)$ $\delta_C: O(1)$

func freqQuery(Q[], A[]) { *extra space*

HashMap < Integer, Integer > mp; A = { 2, 6, 3, 8, 2, 8, 2, 3, 8, 10, 6 }

q = Q.size();

n = A.size();

for (i → 0 to n-1) {

ele = A[i];

if (mp.containsKey(ele)) {

mp.put(ele, mp.get(ele) + 1);

3 else {

mp.put(ele, 1);

3

3

<10, 1>

<8, 3>

<3, 2>

<6, 2>

<2, 3>

mp

2 → 1 + 2 = 3

TC: O(N + Q)

SC: O(N);

for (i → 0 to q-1) {

ele = Q[i];

if (mp.containsKey(ele)) {

print (mp.get(ele),

3 else {

print (0);

1

3

7

Given an array of N elements, find the count of distinct elements.

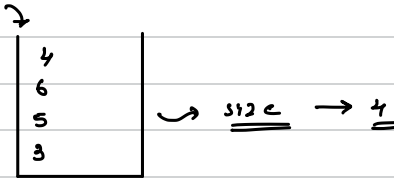
$A = \{ \underline{3}, \underline{5}, \underline{4}, \underline{5}, \underline{4} \}$

total $\rightarrow$ 5 elements

3, 4, 5, 6 $\rightarrow$

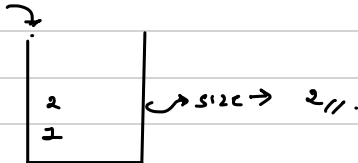
$\rightarrow$

ans = 4



Hash set.

$A = \{ \underline{1}, \underline{1}, \underline{2}, \underline{2}, \underline{1}, \underline{2}, \underline{1}, \underline{2}, \underline{1}, \underline{1}, \underline{1} \}$



HashSet<Integer> hs;
extra space

```
for i  $\rightarrow$  0 to n-1  $\rightarrow$  N  
|   hs.add(A[i]);  
|  
|  
|  
print (hs.size());
```

TC : $O(N)$
SC : $O(N)$

Two Sum

Q. Given $\text{arr}[N]$ and K , check if there exists a pair (i, j) such that,

$$\text{arr}[i] + \text{arr}[j] == K \text{ \&\& } i \neq j$$

$A = \{ 8, 9, -2, 4, 5, 11, -6, 4 \}$ $K = 6$

$\text{arr}[0] + \text{arr}[2] = 8 - 2 = 6 \implies \text{Present.}$

$K = 22$

Not present

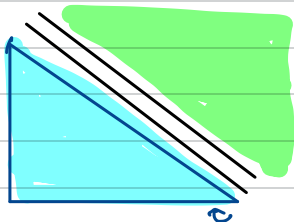
Ques $A = [3, 5, 1, 2, 1, 2]$ and $K = 7$.

True

$K = 10 \rightarrow \underline{\text{false}}$

Brute force

i, j



func checkPair(A, K) {

for ($i \rightarrow 0$ to $n-1$) {

$\longrightarrow N$

for ($j \rightarrow 0$ to $n-1$) {

$\longrightarrow N$

if ($i \neq j$ & $\text{arr}[i] + \text{arr}[j] == K$) {

return true;

}

return false;

}

TC: $O(N^2)$

SC: $O(1)$

extra



optimization $\Rightarrow$ Hashset, HashMap

$$a + b = k.$$

$$a[i] + a[j] = k$$

$k = 9$

	0	1	2	3	4	5	6	7	8
d	8	9	2	-2	4	5	11	-6	4
$k - a[i] =$	1	0	7	11					
	x	x	x						

$\downarrow$
 $\checkmark \rightarrow$ return True

$$a + b = k \rightarrow b = k - a$$

$\curvearrowright 11? \checkmark$

8	9	2
-2	4	5
11	-6	

Hashset

Edge case

$k = 4$

	0	1	2	3	4	5	6	7	8
d	8	9	2	-2	4	5	11	-6	4
$k - a[i] =$	-4	-5	2						
	x	x							

$\downarrow$
 $\checkmark \rightarrow$ return True

$\curvearrowright 2 \checkmark$

8	9	2
-2	4	5
11	-6	

Hashset

wrong $\leftarrow$
 index info
 is missing.

Using Hashset we cannot solve above problem



for each element we store we need index.

$\rightarrow$ $\langle a, \text{index} \rangle$

k = 4

0 1 2 3 4 5 6 7 8
d 8, 9, 4, -2, 2, 5, 11, -6, 4

k - arr[i] =

-4 -5 0 6 2
x x x x x

2

8 9 4 -2

HashSet

Break 10:20 pm → 10:30 pm

func targetSum(A, k) {

HashSet<Integer> hs;

n = A.size();

for i → 0 to n-1 {

if (hs.contains(k - arr[i])) {

return true;

} else {

hs.insert(arr[i]);

} return false;

Count pairs (i, j) such that, $arr[i] + arr[j] == K$ && $i \neq j$ in the given array.

$A = [3, 5, 1, 2, 1, 2]$ and $K = 3$.

Note that (i, j) and (j, i) considered as same.

$arr[2] + arr[3] = 1$

$arr[2] + arr[5] = 2$

$arr[3] + arr[4] = 3$

$arr[4] + arr[5] = 4$

ans = 4//.

$A = [1, 2, 5, 2, 5, 8, 5, 2, 8]$ $k = 10$

ans = 9//.

Dry Run

$A = [1, 2, 5, 2, 5, 8, 5, 2, 8]$ $k = 10$

target = $\begin{matrix} 8 & 5 & 8 & 5 & 2 & 5 & 8 & 2 \\ \times & \times & \times & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \end{matrix}$

count = $\begin{matrix} 0 & 0 & 0 & +1 & +2 & +2 & +1 & +3 \end{matrix} = 9//.$

why adding value to hashmap on the go??

↓
because to avoid $i = j$

8 ✓

$\langle 2, 3 \rangle$

$\langle 5, 3 \rangle$

$\langle 8, 2 \rangle$

HashMap.

pseudo code

```
func targetCount (arr, k) {  
    n = arr.length  
    [HashMap < Integer, Integer > hm;] → extra space  
    c = 0;  
    for i → 0 to n-1 {  
        target = k - arr[i];  
        if (hm.containsKey(target)) {  
            c = c + hm.get(target);  
        }  
        if (hm.containsKey(arr[i])) {  
            hm.put(arr[i], hm.get(arr[i]) + 1);  
        } else {  
            hm.put(arr[i], 1);  
        }  
    }  
    return c;  
}
```

Time: $O(N)$;
Space: $O(N)$;